Lab2 EEG Classification

TA: 良亦＆致雅

Important Rule

Submission Deadline: 11/05 (Thurs.) 23：59

Late submission: grade * 80%

Turn in:

1）Experiment Report (.pdf) to E3 「LAB2_yourstudentID_name.pdf」eg : 「LAB2_313XXXXXX_吳小恩.pdf」

2）Source Code (.py) **to your own github** (No need to upload to E3)

Lab Objective

EEG Classification with BCI competition dataset

Object 1: Implement simple EEG classification model - EEGNet[1]

Object 2: Discuss the depthwise convolution and seperable convolution in the report

Object 3: Evaluation

[1] EEGNet: A Compact Convolutional Neural Network for EEG-based Brain-Computer Interfaces

Requirements

1. Implement the **EEGNet, you should write the architecture yourself**.

2. **Visualize the accuracy and loss trend** during training phase **(necessary)**.

3. **Experiment with different settings** to enhance the model's performance. **Discuss your findings in the report's discussion section.** The more variations you explore, the better your grade.

    - activation functions, ELU alpha values, dropout rates, or others

4. Upload you code to your own github including README.md

Bonus: Implement DeepConvNet or other EEG classification model and discuss your results (15%)

Dataset

- BCI Competition III – IIIb
- [2 classes, 2 bipolar EEG channels]
- *Reference: http://www.bbci.de/competition/iii/desc_IIIb.pdf*

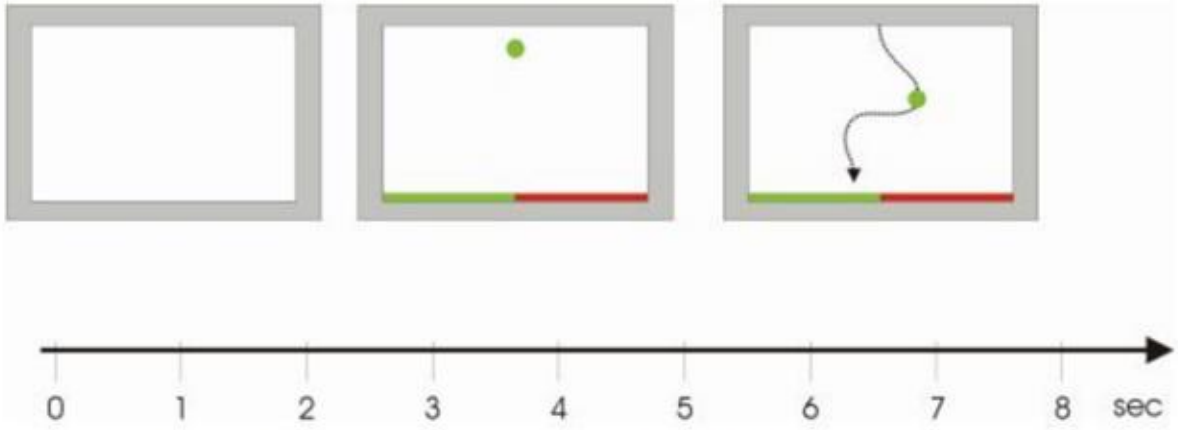


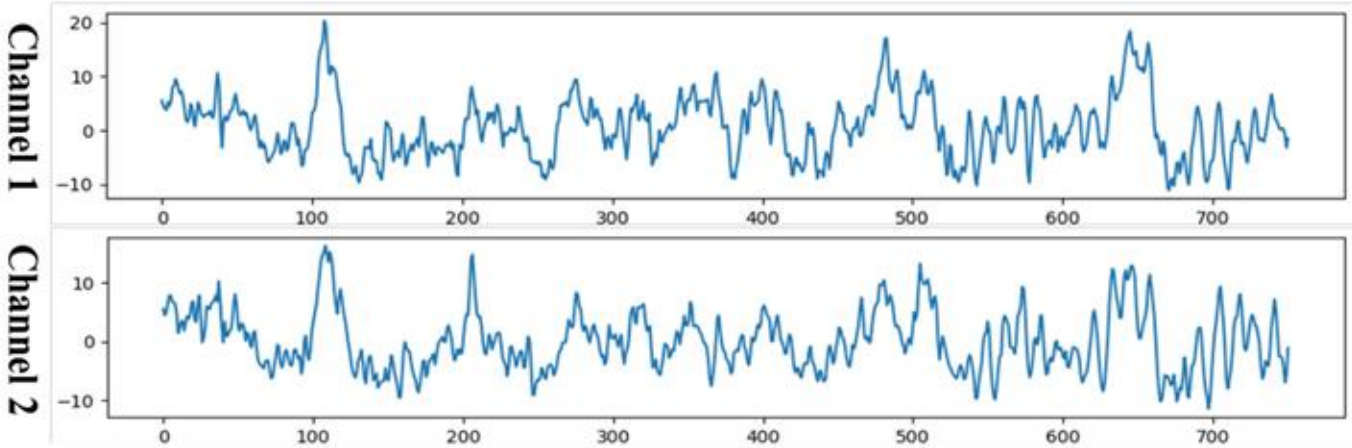
Figure 3: Basket paradigm used for S4 and X11 [3].

Prepare Data

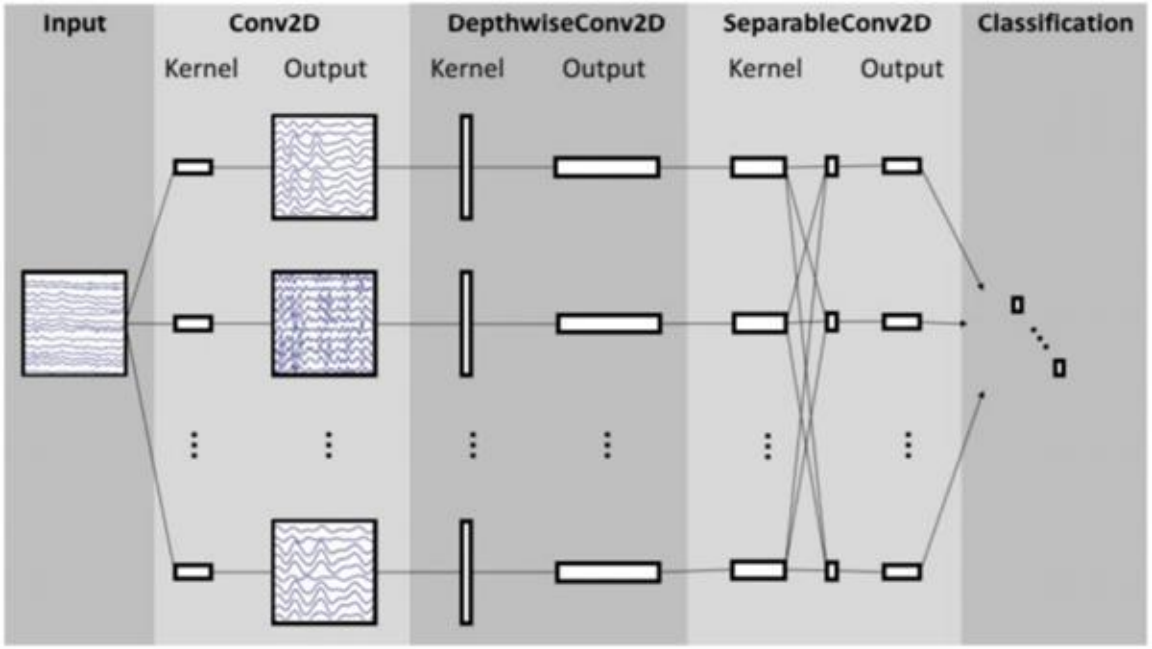
- Training data: S4b_train.npz, X11b_train.npz
- Testing data: S4b_test.npz, X11b_test.npz
- To read the preprocessed data, refer to the “dataloader.py”.

B: batch size

- **Input: [B, 1, 2, 750] Output: [B, 2] Ground truth: [B]**



Create Model - EEGNet



Reference: Depthwise Separable Convolution https://towardsdatascience.com/a-basic-introduction-to-separable-convolutions-b99ec3102728

Create Model - EEGNet

- EEGNet implementation details

```
EEGNet(
  (firstconv): Sequential(
    (0): Conv2d(1, 16, kernel_size=(1, 51), stride=(1, 1), padding=(0, 25), bias=False)
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
  (depthwiseConv): Sequential(
    (0): Conv2d(16, 32, kernel_size=(2, 1), stride=(1, 1), groups=16, bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ELU(alpha=1.0)
    (3): AvgPool2d(kernel_size=(1, 4), stride=(1, 4), padding=0)
    (4): Dropout(p=0.25)
  )
  (separableConv): Sequential(
    (0): Conv2d(32, 32, kernel_size=(1, 15), stride=(1, 1), padding=(0, 7), bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ELU(alpha=1.0)
    (3): AvgPool2d(kernel_size=(1, 8), stride=(1, 8), padding=0)
    (4): Dropout(p=0.25)
  )
  (classify): Sequential(
    (0): Linear(in_features=736, out_features=2, bias=True)
  )
)
```

Create Model - Activation Functions

- In the PyTorch framework, it is easy to implement the activation function.

```
nn.LeakyReLU(),
nn.ReLU(),
nn.ELU(),
```

```
EEGNet(
  (firstconv): Sequential(
    (0): Conv2d(1, 16, kernel_size=(1, 51), stride=(1, 1), padding=(0, 25), bias=False)
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
  (depthwiseConv): Sequential(
    (0): Conv2d(16, 32, kernel_size=(2, 1), stride=(1, 1), groups=16, bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ELU(alpha=1.0)
    (3): AvgPool2d(kernel_size=(1, 4), stride=(1, 4), padding=0)
    (4): Dropout(p=0.25)
  )
  (separableConv): Sequential(
    (0): Conv2d(32, 32, kernel_size=(1, 15), stride=(1, 1), padding=(0, 7), bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ELU(alpha=1.0)
    (3): AvgPool2d(kernel_size=(1, 8), stride=(1, 8), padding=0)
    (4): Dropout(p=0.25)
  )
  (classify): Sequential(
    (0): Linear(in_features=736, out_features=2, bias=True)
  )
)
```

DeepConvNet (Bonus)

Layer	# filters	size	# params	Activation	Options
Input		(C, T)			
Reshape		(1, C, T)			
Conv2D	25	(1, 5)	150	Linear	mode = valid, max norm = 2
Conv2D	25	(C, 1)	25 * 25 * C + 25	Linear	mode = valid, max norm = 2
BatchNorm			2 * 25		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	50	(1, 5)	25 * 50 * C + 50	Linear	mode = valid, max norm = 2
BatchNorm			2 * 50		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	100	(1, 5)	50 * 100 * C + 100	Linear	mode = valid, max norm = 2
BatchNorm			2 * 100		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	200	(1, 5)	100 * 200 * C + 200	Linear	mode = valid, max norm = 2
BatchNorm			2 * 200		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Flatten					
Dense	N			softmax	max norm = 0.5

DeepConvNet (Bonus)

Implement the DeepConvNet architecture by using the table in the previous slide. The parameter C = 2, T = 750 and N = 2. The max norm term is ignorable.

You can read the original architecture table of DeepConvNet in the appendix of the paper I gave you

[1] EEGNet: A Compact Convolutional Neural Network for EEG-based Brain-Computer Interfaces

Hyper Parameters

- Batch size= 64
- Learning rate = 1e-2
- Epochs = 150
- Optimizer: Adam
- Loss function: torch.nn.CrossEntropyLoss()

- **You can adjust the hyper-parameters according to your own ideas.**

- If you use “nn.CrossEntropyLoss”, don’t add softmax after final fc layer because this criterion combines LogSoftMax and NLLLoss in one single class.

How to plot on Python

Tutorial:

https://matplotlib.org/stable/tutorials/introductory/pyplot.html
https://seaborn.pydata.org/generated/seaborn.heatmap.html

Building Models with PyTorch

Tutorial:

https://pytorch.org/tutorials/beginner/introyt/modelsyt_tutorial.html

https://www.youtube.com/watch?v=tHL5STNJKag&ab_channel=RobMulla

(Start from 14:02)

Report Spec

1. Introduction (20%)

    a. Introduce EEGNet

2. Experiment setups and result (35%)

    a. All of your experiments and results
        i. Compare different settings you have done with tables or plots
    b. Highest testing accuracy **(screenshot)**
    c. Ploting

        i. Train/ Test loss and accuracy curve

    d. Anything you want to present

Report Spec

3. Discussion (40%) **(Most important part)**

   a. Discuss your findings or share anything you want to share

   b. Explain how you implement the model

   c. Explain the ELU function

   d. Explain what is the depthwise convolution and seperable convolution

4. Github Link (5%) (Do not forget)

5. Bonus (15%): Remember to show your accuracy and show anything you want to present

Score criterion of Lab2

Score: 20% Performance + 80% Report

P.S **If the report exists format errors (file name or the report spec), it will be 5 points penalty (-5)**

---- Criterion of result (20%) ----

Accuracy > = 87% → 100 pts

Accuracy 85~87% → 90 pts

Accuracy 80~85% → 80 pts

Accuracy 75~80%→ 70 pts

Accuracy < 70% → 60 pts